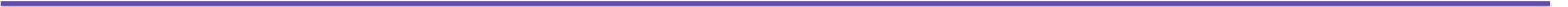


Lecture 6

From Project Idea to First Prototype



Where We Are Now

- Project selected and refined
- Team Charter completed
- Mentor meetings underway
- Now: move from planning → building and testing



Focus for This Week

- Demo -1: show concrete early progress
 - Early Teamwork Survey
 - Equipment / Software / Service Request
 - Start technical development
 - Begin preparing for Presentation 1
-

Demo -1: Purpose

- An early progress checkpoint — not a finished product
- Show evidence that your project is moving from idea to implementation
- Different projects may show different kinds of progress
- Use feedback to decide your next technical step



What Can You Show for Demo -1?

Software / Data

- Early prototype or working component
- UI mockup or interaction flow
- API/service test
- Dataset exploration or pipeline

Algorithms / Hardware

- Algorithm experiment
 - Model or baseline test
 - Hardware proof of concept
 - Architecture + one working piece
-

Demo -1: What We Are Looking For

- Concrete progress since the project proposal
 - Clear explanation of what you tried
 - Evidence: code, test, mockup, data, experiment, or working component
 - What worked / what did not work / what you learned
 - A realistic next step
-

A Good Demo -1

- Show — do not only describe
 - Keep the demo focused on the most important progress
 - Explain technical choices in plain language
 - It is okay if something does not work yet
 - Be ready to discuss what you will do next
-

Demo -1 Grading

- Use the Demo -1 rubric provided by the instructors
 - Focus on demonstrated progress, clarity of the demo, and understanding of the work
 - The goal is meaningful early progress — not a polished final product
-

Requesting Project Resources

**Reuse /
borrow**

**Free /
student
resources**

**Submit
request**

**Instructor
review**

**Purchase if
needed**

Equipment / Software / Service Request

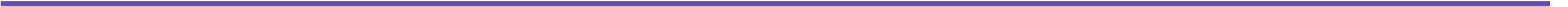
- Submit only if your team has a genuine project need
 - Examples: hardware, software/license, API/AI service, cloud/computing service
 - Check reusable equipment and free/student options first
 - Do not purchase anything before instructor approval
 - Form link: see the course website
-

Early Teamwork Survey

- Individual submission: evaluate your three Senior Design teammates
 - Focus: contribution, communication, reliability
 - Confidential, but not anonymous
 - Individual ratings/comments are not shared directly with teammates
 - One-person Senior Design teams collaborating with ME/ECE/etc. do not submit
 - Form link: see the course website
-

Your Team Charter Is a Living Document

- Are roles actually working?
- Is communication working?
- Is workload reasonably distributed?
- Are decisions being made effectively?
- If something is not working, discuss it and adjust your team practices



Moving Into Development

Project idea

**Technical
investigation**

Prototype

Test

Revise



Sprint / GitHub Project Expectations

- Create meaningful tickets for project work
 - Assign owners and keep status current
 - Post weekly status updates
 - Link code / pull requests to tickets when appropriate
 - Close, move, or revise work as the sprint evolves
-

Git Workflow for Senior Design

**Start from
main**

**Create
feature
branch**

**Make
focused
changes**

Open PR

**Review +
merge**



Feature Branch → Pull Request → Main

- Do not push feature work directly to main
 - Keep each branch / PR focused on one piece of functionality
 - Explain what changed and why
 - Link the PR to the related sprint ticket when possible
 - Review each other's code
-

Good Pull Requests

- Small and focused is better than one very large PR
 - Describe WHAT changed and WHY
 - Include enough testing information for a reviewer
 - Resolve important feedback before merging
 - Use reviews to improve shared understanding of the codebase
-

CI/CD: Why We Care

- Integrate changes frequently
- Catch problems early
- Automate repetitive checks
- Make builds and deployments more reliable
- Start simple — improve the pipeline as the project grows



CI/CD for Senior Design

- GitHub Actions is a practical starting point
 - Useful CI steps: lint → test → build
 - Deployment can be added when appropriate for your project
 - A minimal pipeline is enough to start
 - Do not over-engineer before you have a working prototype
-

Looking Ahead: Presentation 1

- Next step: explain why your project matters to a nontechnical audience
 - Build from your project proposal, but make the story clearer and more persuasive
 - Connect the problem → users → solution → value
 - Full presentation preparation will continue next week
-

Presentation 1: Elevator Pitch

- 4 minutes + 2 minutes for questions
 - Audience: nontechnical — e.g., investors or upper management
 - What are you building and why?
 - Who are the users?
 - How is it different from current products or research?
 - Presentation skills matter — practice explaining the idea clearly
-

Start Preparing Now

- Refine the problem and user story
- Simplify technical language
- Choose visuals that help the audience understand
- Reuse useful material from the project proposal — but improve it
- Practice a short explanation of why the project is worth building



Before Next Week

- Complete Demo -1
 - Complete the Early Teamwork Survey
 - Submit a resource request only if needed
 - Continue meeting with your mentor
 - Keep GitHub project / weekly updates current
 - Start preparing for Presentation 1
-

Course Website

- Use the course website as the source of truth for deadlines, assignments, rubrics, and links
- <https://gw-cs-sd-26-27.github.io/>



Technical Reference

Optional slides for later review



Reference: Developing a Feature

- Pull the latest main branch
- Create a feature branch
- Make and commit changes
- Bring main into your branch if needed
- Push branch and open a pull request



Reference: PR Review Mindset

- Goal: improve the codebase, not demand perfection
 - Block only when an issue truly prevents safe/appropriate merging
 - Use comments to discuss design and implementation
 - Small feedback can become future work rather than blocking progress
-

Reference: Code Expectations

- All students should contribute code during the development sprint
- Use feature branches and pull requests
- Link PRs to tickets when possible
- Only code merged to main represents integrated project progress



Reference: Weekly Status Updates

- What did you complete?
 - What are you currently working on?
 - What is blocking you?
 - Keep the update concise and specific
 - Each student should make their own contribution visible
-

Reference: CI/CD Starter Checklist

- Repository builds successfully
 - Automated linting where appropriate
 - Automated tests where appropriate
 - Build validation on pushes / pull requests
 - Deployment automation only when useful for your project
-

Reference: Demo 0 Comes Later

- Demo -1 = early team/project progress checkpoint
- Demo 0 = later individual progress and code review with your instructor
- Use the current sprint to build toward Demo 0

